Machine Learning

(Course Code: 18B1WCI634/18B11BI611)

Decision Trees

Mr. Sandeep Kumar Patel

Assistant Professor

Jaypee University of Information Technology

Waknaghat, Solan, HP-173234

What is a Decision Tree?

- A **Decision Tree** is a tree-structured model used for**Classification** (predicting categorical labels) and **Regression** (predicting continuous values).
- It is a **non-parametric supervised learning method** that learns decision rules directly from training data.
- Formally, a decision tree represents a function:

$$h : X \to Y$$

where:

- X = input feature space
- Y = output space (class label or numeric value)

Decision Tree Terminology

- **Root Node**: The top node representing the entire dataset.
- **Decision (Internal) Node**: A node where a feature is tested.
- **Leaf Node**: The final node that gives the prediction (class or value).
- **Branch**: A connection showing the result of a decision.
- **Path**: A sequence of decisions from the root to a leaf.
- **Parent and Child Nodes**: A parent splits into one or more child nodes.
- **Subtree**: A smaller part of the tree.
- **Pruning**: Removing parts of the tree to avoid overfitting.

Decision Tree Example

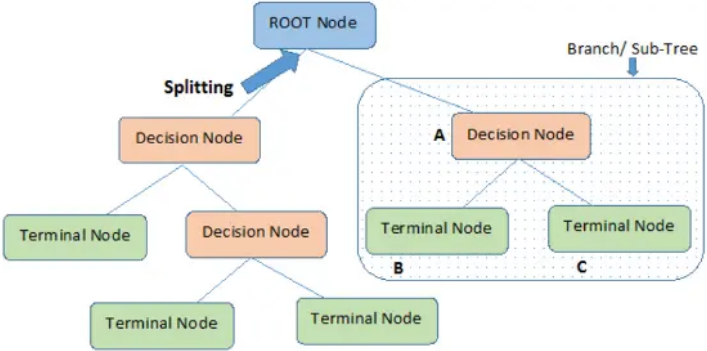


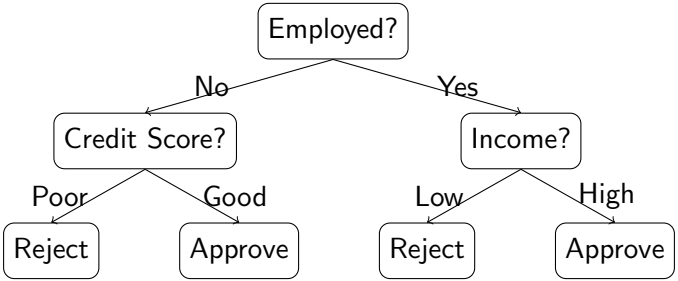
Figure: Basic Decision Tree Example

When is Decision Tree Learning Appropriate?

Decision Tree learning is suitable when:

- Data is represented in **attribute–value** format
  Example: Outlook = Sunny, Temperature = Hot, Humidity = High
- The target function is **discrete-valued** (classification problems)
  Example: Spam / Not Spam, Yes / No, Disease / No Disease
- The concept can be expressed as **if–then rules** (disjunctive concepts)
  Example: If (Outlook = Sunny AND Humidity = High) then Play = No
- The dataset may contain **noise**
  Example: Incorrect class labels or measurement errors
- The dataset may contain **missing attribute values**
  Example: Temperature value not recorded for some instances

Decision Tree Example:Loan Approval



Loan Approval Example: Concept Identification

1. Attribute–Value Representation

Each applicant is described using attributes:

$$x = (\text{Employed}, \text{Credit Score}, \text{Income})$$

Attribute–Value Pairs:

- (Employed = Yes), (Employed = No)
- (Credit Score = Good), (Credit Score = Poor)
- (Income = High), (Income = Low)

2. Target Function

$$h : X \to Y$$

$$Y = \{\text{Approve}, \text{Reject}\}$$

Target Concept and Disjunctive Function

3. Target Concept (Set of Rules)

- If (Employed = No AND Credit Score = Poor) $\rightarrow$ Reject
- If (Employed = No AND Credit Score = Good) $\rightarrow$ Approve
- If (Employed = Yes AND Income = Low) $\rightarrow$ Reject
- If (Employed = Yes AND Income = High) $\rightarrow$ Approve

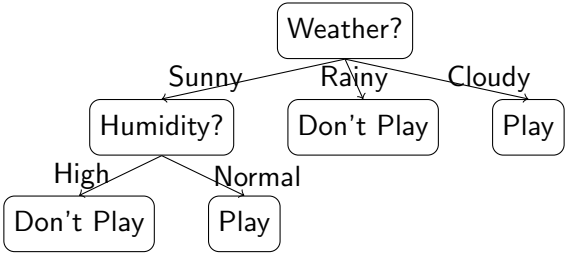
4. Disjunctive Function Representation

Approve class can be written as:

$$(Employed = No \wedge Credit = Good) \ \vee \ (Employed = Yes \wedge Income = High)$$

Decision trees represent a disjunction (OR) of conjunctions (AND conditions).

Decision Tree with Three Branches (Weather Example)



Weather Example: Concept Identification

1. Attribute–Value Representation

Each instance is represented as:

$$x = (\text{Weather}, \text{Humidity})$$

Attribute–Value Pairs:

- (Weather = Sunny), (Weather = Rainy), (Weather = Cloudy)
- (Humidity = High), (Humidity = Normal)

2. Target Function

$$h : X \to Y$$

$$Y = \{\text{Play}, \text{Don't Play}\}$$

Target Concept and Disjunctive Form

3. Target Concept (Set of Rules)

- If (Weather = Sunny AND Humidity = High) $\rightarrow$ Don't Play
- If (Weather = Sunny AND Humidity = Normal) $\rightarrow$ Play
- If (Weather = Rainy) $\rightarrow$ Don't Play
- If (Weather = Cloudy) $\rightarrow$ Play

4. Disjunctive Representation

Play class:

$$(Weather = Sunny \wedge Humidity = Normal) \ \vee \ (Weather = Cloudy)$$

Don't Play class:

$$(Weather = Sunny \wedge Humidity = High) \ \vee \ (Weather = Rainy)$$

Decision Tree Hypothesis Space and Model Selection

- For a given training dataset, **many different decision trees** may exist that correctly classify the training examples.
- These trees may differ in:
  - Tree depth
  - Number of nodes
  - Choice and order of splitting attributes
- All possible decision trees together form the **hypothesis space** H.
- However, not every tree is desirable:
  - Some trees may become **very deep and complex**
  - They may **overfit the training data**
  - Overfitting leads to **low training error but poor performance on unseen data**
- **Goal:** Select the **best decision tree** that
  - Achieves high predictive accuracy
  - Generalizes well to new data

How to Build the Best Tree?

Decision tree construction uses a greedy, top-down approach:

- Start at the root node with the full dataset
- At each node:
  - Evaluate all candidate attributes
  - Measure the **impurity/uncertainty before and after splitting**
  - Select the attribute that **maximizes impurity/uncertainty reduction**
- Apply recursively on each child node
- Apply **pruning** if necessary to reduce overfitting

When to Split a Node

Split a node if:

- The node contains **examples from multiple classes** (impure)
- There are **remaining attributes** to split on
- Splitting improves the **homogeneity of child nodes** (reduces impurity)

Stop splitting (create a leaf) if:

- All examples belong to the **same class** $\rightarrow$ pure node
- No remaining attributes are available
- Node has **too few examples** (prevents overfitting)
- Further splitting does not significantly reduce impurity

Which Attribute to Choose?

Different decision tree algorithms select the best attribute using different criteria:

- **ID3**
  - Uses **Entropy** to measure impurity
  - Chooses attribute with highest **Information Gain (IG)**
- **C4.5**
  - Extension of ID3
  - Uses **Gain Ratio** to reduce bias toward multi-valued attributes
  - Handles **continuous attributes** and **missing values**
  - Supports **post-pruning** to avoid overfitting

Summary: Building the Best Decision Tree

- Start with the full dataset at the root
- Recursively select the **best attribute** at each node
- Split nodes until stopping criteria are met (purity, attributes, sample size)
- Optionally prune the tree to prevent overfitting
- Result: A tree that balances **accuracy, simplicity, and generalization**

ID3 Algorithm

Steps of the ID3 Algorithm:

- Compute the **entropy** of the entire dataset.
- For each attribute:
  - Partition the dataset based on attribute values
  - Compute the **Information Gain**
- Select the attribute with the **highest Information Gain** as the root node.
- Recursively repeat the process for each subset using remaining attributes.
- Stop when:
  - All examples belong to the same class, or
  - No attributes remain.

Entropy: Measuring Impurity/Uncertainty

For a dataset S containing positive and negative examples:

$$Entropy(S) = -p_+ \log_2 p_+ - p_- \log_2 p_-$$

- p_+ = proportion of positive examples in S
- p_- = proportion of negative examples in S

Interpretation:

- $Entropy = 0 \rightarrow$ Pure node (all examples belong to one class)
- $Entropy = 1 \rightarrow$ Maximum impurity (50%-50% split)
- $0 < Entropy < 1 \rightarrow$ Mixed class distribution

Information Gain

For attribute A applied to dataset S:

$$Gain(S, A) = Entropy(S) - \sum_{v \in Values(A)} \frac{|S_v|}{|S|} Entropy(S_v)$$

- S_v = subset of S where attribute $A = v$
- $|S_v|/|S|$ = weight of subset

Decision Rule: Select the attribute with the **highest Information Gain**.

Training Dataset: Play Tennis Example

Outlook	Temp	Humidity	Wind	Play
Sunny	Hot	High	Weak	No
Sunny	Hot	High	Strong	No
Overcast	Hot	High	Weak	Yes
Rain	Mild	High	Weak	Yes
Rain	Cool	Normal	Weak	Yes
Rain	Cool	Normal	Strong	No
Overcast	Cool	Normal	Strong	Yes
Sunny	Mild	High	Weak	No
Sunny	Cool	Normal	Weak	Yes
Rain	Mild	Normal	Weak	Yes
Sunny	Mild	Normal	Strong	Yes
Overcast	Mild	High	Strong	Yes
Overcast	Hot	Normal	Weak	Yes
Rain	Mild	High	Strong	No

Step 1: Entropy of the Parent Node

Total examples: $|S| = 14$

$$S = 9 \text{ Yes}, \quad 5 \text{ No}$$

$$p_+ = \frac{9}{14}, \quad p_- = \frac{5}{14}$$

$$Entropy(S) or H(S) = -\frac{9}{14}\log_2\frac{9}{14} - \frac{5}{14}\log_2\frac{5}{14}$$

$$H(S) \approx 0.940$$

Interpretation: The dataset is moderately impure.

Step 2: Information Gain for Outlook

Outlook values: Sunny, Overcast, Rain

- Sunny (2 Yes, 3 No)

$$Entropy(Sunny) \approx 0.971$$

- Overcast (4 Yes, 0 No)

$$Entropy(Overcast) = 0$$

- Rain (3 Yes, 2 No)

$$Entropy(Rain) \approx 0.971$$

$$Entropy_{split}(Outlook) = \frac{5}{14}(0.971) + \frac{4}{14}(0) + \frac{5}{14}(0.971) \approx 0.693$$

$$Gain(S, Outlook) = 0.940 - 0.693 \approx 0.247$$

Step 2 (cont.): Compare Other Attributes

Information Gain values:

- Temperature: $Gain(S, Temperature) \approx 0.029$
- Humidity: $Gain(S, Humidity) \approx 0.151$
- Wind: $Gain(S, Wind) \approx 0.048$

Highest Gain: Outlook (0.247)

Therefore, Root Node = Outlook

Step 2 (cont.): Compare Other Attributes

- **Overcast Branch:** Since Entropy was 0, this becomes a Leaf Node labeled "Yes". (If it's overcast, we always play).
- **Sunny Branch:** We still have a mix of Yes/No. We repeat the ID3 process only for the rows where Outlook = Sunny. (Usually, Humidity wins here).
- **Rain Branch:** We repeat the process for the Rain rows. (Usually, Wind wins here).

Subset: S_{sunny} (Outlook = Sunny)

Outlook	Temp	Humidity	Wind	Play
Sunny	Hot	High	Weak	No
Sunny	Hot	High	Strong	No
Sunny	Mild	High	Weak	No
Sunny	Cool	Normal	Weak	Yes
Sunny	Mild	Normal	Strong	Yes

Class Distribution: 3 No, 2 Yes

Starting Entropy:

$$H(S_{sunny}) = -\left(\frac{2}{5}\log_2\frac{2}{5} + \frac{3}{5}\log_2\frac{3}{5}\right) \approx \mathbf{0.971}$$

Information Gain: Humidity

Humidity Values:

- High (3 No) $\Rightarrow H = 0$
- Normal (2 Yes) $\Rightarrow H = 0$

$$IG(S_{sunny}, Humidity) = 0.971 - \left[\frac{3}{5}(0) + \frac{2}{5}(0)\right] = \mathbf{0.971}$$

Perfect Split

Information Gain: Wind and Temp

Wind

- Weak (1 Yes, 2 No) $\Rightarrow H = 0.918$
- Strong (1 Yes, 1 No) $\Rightarrow H = 1$

$$IG(S_{sunny}, Wind) \approx \mathbf{0.02}$$

Temp

- Hot (2 No) $\Rightarrow H = 0$
- Mild (1 Yes, 1 No) $\Rightarrow H = 1$
- Cool (1 Yes) $\Rightarrow H = 0$

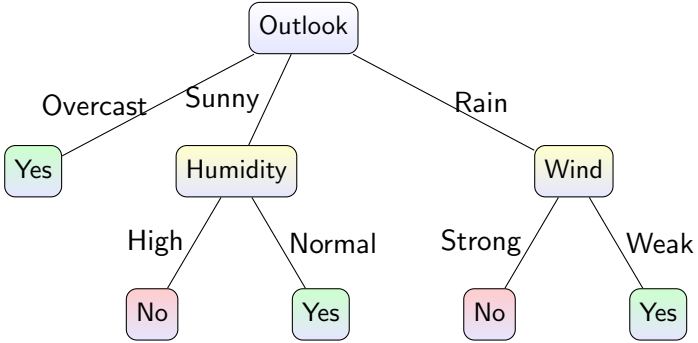
$$IG(S_{sunny}, Temp) = \mathbf{0.571}$$

Final Comparison

Attribute	Information Gain
Humidity	**0.971**
Temp	0.571
Wind	0.02

Humidity is selected for the Sunny branch.

Decision Tree Constructed by ID3



Limitations of ID3 Algorithm

- **Maintains Only One Hypothesis**
  - ID3 keeps only one decision tree during learning.
  - It does not track all possible consistent hypotheses like the **Candidate-Elimination algorithm**.
- **No Backtracking**
  - Once ID3 selects an attribute for a split, it never revisits that decision.
  - This may lead to **local optimum** solutions instead of the best possible tree.
- **Greedy Search**
  - ID3 uses a hill-climbing (greedy) strategy.
  - It may miss better trees in other regions of the search space.
- **Sensitive to Overfitting**
  - Large trees may fit noise in the training data.
  - This problem can be reduced using **pruning techniques**.

Inductive Bias in Decision Tree (ID3)

Inductive Bias:// Inductive bias refers to the set of assumptions a learning algorithm uses to predict outputs for unseen inputs.

- Training data alone is not sufficient to determine a unique hypothesis.
- Bias guides the algorithm in selecting a particular hypothesis.

Inductive Bias of ID3:

- ID3 prefers **shorter decision trees or hypothesis** over longer ones.
- ID3 prefers Attributes with **high information gain** are placed near the root.

Why Prefer Short Hypotheses?

Occam's Razor Principle

Among competing hypotheses that explain the data equally well, Prefer the **simplest hypothesis** that fits the training data.

Why Prefer Simple Hypotheses?

- There are fewer simple hypotheses than complex ones. Simple models are less likely to fit data by coincidence. i.e. Complex hypotheses are more likely to fit noise in the training data.
- Simpler hypotheses are easier to compute and learning algorithms can search them more efficiently.
- simpler models tend to generalize better to unseen data i.e. reduce the risk of **overfitting**.

Overfitting Definition

A hypothesis h is said to **overfit the data** if there exists another hypothesis h' such that:

- h has **lower error on training data** than h'.
- But h has **higher error on the true distribution** (test/unseen data) than h'.

Formal Definition

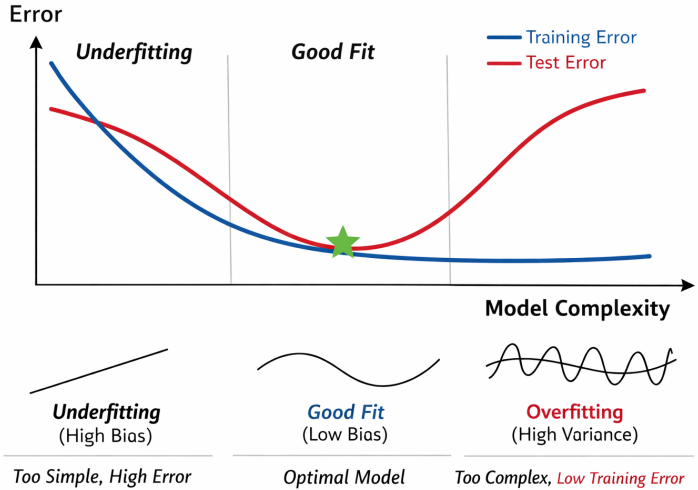
A hypothesis $h \in H$ overfits the training data if there exists $h' \in H$ such that

$$error_{train}(h) < error_{train}(h')$$

but

$$error_{test}(h) > error_{test}(h')$$

Overfitting



Why Overfitting Occurs & effects

Overfitting may occur due to:

- Noise or errors in the training data.
- Small training datasets.
- Coincidental patterns in the data.

These factors can cause the decision tree to grow too complex.

Effect of Overfitting

- As the decision tree grows larger:
  - Training accuracy increases
  - Test accuracy first increases then decreases
- Very large trees may memorize training data instead of learning real patterns.

Avoiding Overfitting in Decision Trees

- **Pre-Pruning (Early Stopping):** Stop the tree growth early by setting limits such as maximum depth or minimum samples per node.
- **Post-Pruning:** First grow the full tree and then remove branches that do not improve performance on validation data.

Pre-Pruning in Decision Trees

Pre-Pruning (Early Stopping)

Pre-pruning stops the growth of the decision tree early to prevent overfitting.

- Stop splitting when the **information gain is not statistically significant** using Chi-square test.
- Set a **maximum depth** for the tree.
- Apply the **Minimum Description Length (MDL)** principle to avoid overly complex trees.
- Require a **minimum number of samples** in a node before splitting.
- Stop if the **information gain is below a threshold**.

Post-Pruning in Decision Trees

Post-Pruning

Post-pruning first builds the full decision tree and then removes branches that do not improve performance on validation data.

Reduced Error Pruning

Reduced Error Pruning is a **post-pruning technique** used to avoid overfitting in decision trees by removing branches that do not improve accuracy on a validation dataset.

Procedure

- Split the available data into **training set** and **validation set**.
- Grow the decision tree completely using the training data.
- For each non-leaf node, consider replacing the subtree with a **leaf node**.
- If the replacement **does not decrease validation accuracy**, prune the subtree.
- Repeat the process until no further pruning improves performance.

Rule Post-Pruning Algorithm

Input: A fully grown decision tree and training dataset
Output: Simplified rule set

Algorithm Steps

1. Generate a complete decision tree from the training data.
2. Convert each path from the root node to a leaf node into an IF–THEN rule.
3. For each rule:
    1. Evaluate the rule on a validation dataset.
    2. Remove one condition at a time from the rule.
    3. Recalculate the rule accuracy or error rate.
    4. If removing the condition improves or does not significantly reduce accuracy, prune the condition.
4. Repeat the process until no further pruning improves the rule.
5. Output the final set of simplified rules.

Numerical Problem: Rule Post-Pruning

A rule generated from a decision tree is:

Rule

IF Age = Young AND Student = Yes AND Income = High
THEN Buy_Computer = Yes

Dataset results:

Condition Satisfied	Correct Predictions	Incorrect Predictions
Yes	18	6

Questions

1. Calculate the accuracy of the rule.
2. If removing `Income = High` gives the following results:

Condition Satisfied	Correct Predictions	Incorrect Predictions
Yes	20	7

Should the condition `Income = High` be pruned? Justify your answer.

Solution

Accuracy formula:

$$Accuracy = \frac{Correct}{Correct + Incorrect}$$

Original rule:

$$Accuracy = \frac{18}{18+6} = \frac{18}{24} = 0.75 = 75\%$$

After removing `Income = High`:

$$Accuracy = \frac{20}{20+7} = \frac{20}{27} \approx 0.74 = 74\%$$

Decision

Since the accuracy decreases from 75% to 74%, the condition `Income = High` should **not be pruned**. The original rule should be retained.

Feature	Reduced Error Pruning	Rule Post-Pruning
Pruning Level	Tree-level pruning	Rule-level pruning
When Applied	After the full decision tree is built	After converting tree paths into rules
Method	Removes subtrees if accuracy on validation set does not decrease	Removes conditions from rules if accuracy does not decrease
Data Used	Requires a separate validation dataset	Uses validation or training data to evaluate rule performance
Process	Replace a subtree with a leaf node if it improves validation accuracy	Simplify rules by removing unnecessary conditions
Complexity	Simpler but may remove useful structure	More flexible and produces simpler rules
Output	A simplified decision tree	A simplified rule set

Table: Comparison between Reduced Error Pruning and Rule Post-Pruning

Why C4.5 is Needed

Limitation of ID3	**C4.5 Improvement**
Handles only discrete attributes	Can handle **continuous attributes** using threshold splits
Cannot handle missing attribute values	Assigns **probabilities or fractional counts** to missing values
Information Gain biased toward attributes with many values	Uses **Gain Ratio** to reduce bias
Susceptible to overfitting	Performs **post-pruning** to improve generalization

C4.5 Decision Tree Algorithm

C4.5 is an extension of the ID3 algorithm used for generating decision trees. It selects the best attribute using the **Gain Ratio** instead of Information Gain.

1. Compute the **Entropy** of the dataset
2. For each attribute, compute the **Information Gain**
3. Compute the **Split Information**

$$SplitInfo(S, A) = -\sum_{v \in Values(A)} \frac{|S_v|}{|S|} \log_2 \frac{|S_v|}{|S|}$$

4. Compute the **Gain Ratio**

$$GainRatio(S, A) = \frac{Gain(S, A)}{SplitInfo(S, A)}$$

5. Select the attribute with the **highest Gain Ratio** as the root node.
6. Repeat the process recursively until all samples in a node belong to the same class.

Step 1: Compute Entropy of Parent Node

Dataset: 14 examples (`PlayTennis` = 9 Yes, 5 No)

$$Entropy(S) = -p_+ \log_2 p_+ - p_- \log_2 p_-$$

$$p_+ = \frac{9}{14}, \quad p_- = \frac{5}{14}$$

$$Entropy(S) = -\frac{9}{14} \log_2 \frac{9}{14} - \frac{5}{14} \log_2 \frac{5}{14} \approx 0.940$$

Interpretation: The dataset has moderate impurity.

Step 2: Compute Gain Ratio for Attributes

Example: Attribute Outlook (Sunny, Overcast, Rain)

- Sunny: 2 Yes, 3 No $\rightarrow$ $Entropy(Sunny) \approx 0.971$
- Overcast: 4 Yes, 0 No $\rightarrow$ $Entropy(Overcast) = 0$
- Rain: 3 Yes, 2 No $\rightarrow$ $Entropy(Rain) \approx 0.971$

Weighted entropy for split:

$$Entropy_{split}(Outlook) = \frac{5}{14} \cdot 0.971 + \frac{4}{14} \cdot 0 + \frac{5}{14} \cdot 0.971 \approx 0.693$$

Information Gain:

$$Gain(S, Outlook) = 0.940 - 0.693 \approx 0.247$$

Split Information:

$$SplitInfo(S, Outlook) = -\left(\frac{5}{14}\log_2\frac{5}{14} + \frac{4}{14}\log_2\frac{4}{14} + \frac{5}{14}\log_2\frac{5}{14}\right) \approx 1.577$$

Step 3: Compute Gain Ratio for Other Attributes

- Temperature: Hot, Mild, Cool $\rightarrow$ Gain Ratio ≈ 0.029
- Humidity: High, Normal $\rightarrow$ Gain Ratio ≈ 0.151
- Wind: Weak, Strong $\rightarrow$ Gain Ratio ≈ 0.048

Attribute with highest Gain Ratio: `Outlook`
Root Node = Outlook

Step 4: Build C4.5 Decision Tree

Split dataset by `Outlook`:

- **Overcast** $\rightarrow$ All Yes $\rightarrow$ Leaf = Yes
- **Sunny** $\rightarrow$ 5 examples (2 Yes, 3 No) Next best attribute: **Humidity**
  - Humidity = High $\rightarrow$ No
  - Humidity = Normal $\rightarrow$ Yes
- **Rain** $\rightarrow$ 5 examples (3 Yes, 2 No) Next best attribute: **Wind**
  - Wind = Strong $\rightarrow$ No
  - Wind = Weak $\rightarrow$ Yes

Result: Final C4.5 decision tree handles discrete attributes and can prune if needed.

Handling Missing Values

Sometimes training data may contain **unknown or missing attribute values**. Two common strategies are used:

- **Most Common Value Method:**
  - Replace the missing value with the most common value of that attribute among training examples.
  - Alternatively, use the most common value among examples with the same class.
- **Probabilistic Method (Used in C4.5):**
  - Assign probabilities based on observed frequencies.
  - Example: If $A = 1$ occurs 60% and $A = 0$ occurs 40%, distribute the instance as:

$$0.6 \rightarrow A = 1, \quad 0.4 \rightarrow A = 0$$

  - These fractional instances are used to compute information gain.

Handling Attributes with Differing Costs

In some tasks, attributes have different **costs** (e.g., medical tests).

- Prefer attributes with **lower cost** when possible.
- Modify attribute selection measure:

$$\frac{Gain(S, A)}{Cost(A)}$$

- This biases the decision tree toward **low-cost attributes** while maintaining accuracy.

Thank You ... !!!